

Universe Node: A Protocol for Deterministic Generation of Shared Realities

Ozalune

Abstract. *Universe Node (protocol name: Shared Reality Protocol) is a specification for generating, persisting, and sharing simulated realities — entire universes — from a single numeric seed. It is not a game, a graphics engine, or a file format: it is a set of rules that any implementation, in any language, can follow so that two independent instances arrive at exactly the same result, without ever having exchanged a single piece of information beyond a number. This document describes the architecture of the protocol itself: the principles that support it, the deterministic cascading generation mechanisms, the immutable history model, and the portability model between instances.*

It does not describe any specific application built on top of the protocol; applications (games, simulations, visualization tools) are clients — windows through which someone observes and acts upon the reality the protocol describes.

1. Fundamental Principle

The protocol starts from a very strict separation of responsibilities:

The universe is the reality. The protocol is the language through which that reality is described and modified. The Node is the machine that maintains and participates in that reality. The client is a window through which someone observes and attempts to act upon it.

The protocol has no knowledge of games, genres, or types of fiction. It knows only seven primitives: Entity, Property, Relation, Action, Event, Rule, State. Anything an implementation wants to represent — a planet, a creature, a currency, a character — needs to be expressed in these terms. This deliberate restriction is what makes the protocol generic enough to sustain, at the same time, a science-fiction client and a fantasy client, without one invalidating the other and without the protocol needing to know what “magic” or “science” even mean.

1.1 Why a Protocol, Not a Game Engine

A game engine assumes a genre. It decides, in advance, that “characters,” “enemies,” “items” exist — categories that make sense for that specific game, but that mean nothing outside of it. Universe Node inverts this logic: it defines no content category at all. It defines only how any content, whatever it may be, must behave to be part of a consistent, shared reality.

This has an important practical consequence: two completely different applications, written by different people, can share the same universe — as long as both correctly follow the protocol. One might render that universe as an explorable solar system; another might render it as an abstract board. Both agree on what exists; they freely disagree on how it is shown.

2. Physical Substrate vs. Narrative Layer

This is the rule that underlies the separation described above, and is perhaps the single most important design decision in the entire protocol.

2.1 The Physical Substrate (mandatory, generic)

Every Event that alters the universe's State must be expressible in generic physical/mechanical terms: energy release, matter transfer, property change, creation or destruction of an Entity, change of Relation.

These are the universe's Rules. They have no opinion about *why* something happened — only about *what* changes, and under which conditions the change is permitted.

A conceptual Rule, for example:

Rule: `energy_release`

If an Event of type `"region.energy_release"` occurs with magnitude `M` within a radius `R` from a point:

- every Entity within radius `R` whose `"structural_integrity"` property is less than `M` loses its `"exists_in"` Relation with the current region (i.e., is destroyed/removed from the State)
- the affected region has its `"energy_signature"` property altered

This Rule is blind to genre. It has no idea whether the energy came from a stellar explosion or a spell.

2.2 The Narrative Layer (optional, free, non-normative)

Every Event can carry an optional `narrative` field — free text, defined by the client that generated the action, describing how that event is interpreted within that specific client's genre/fiction.

The same Event type, generated by a fantasy client, could carry `"narrative": "magic_meteor"` — same physical Rule, same real consequence to the State, different story.

Golden rule: the `narrative` field can NEVER be used by a Node to decide whether an Action is valid, or what an Event does to the State. It is purely presentational. A Node that completely ignores every `narrative` field in the universe must still produce a State identical to that of a Node that displays them.

2.3 Breaking the Rules Is Also Ruled

A universe may allow Events that violate “normal” physical behavior — but this is never an informal exception: it is a special, explicit Rule, with its own conditions and its own cost. When this Rule is triggered, it produces a recognizable Event type (`anomaly.created`), which becomes a permanent fact of the universe, visible to any client — never a hidden local effect within a single application.

There is no “cheating physics for free.” Every violation becomes shared, recorded history.

3. The Seven Primitives

The entire protocol is built on seven concepts. No specific content (planet, creature, item) is a primitive — everything is expressed by combining these seven:

Entity — something that exists in the universe. The protocol does not need to know what a specific type “means” narratively.

Property — an attribute of an Entity, modeled as a separate fact (not embedded directly in the Entity), so that Property changes are themselves traceable as Events.

Relation — a link between two Entities (e.g., a planet “orbits” a star).

Action — what an actor (player, system, another Node) requests to happen. An Action never alters the State directly; it is submitted to the validator.

Event — the result of a successfully validated Action. Events are immutable and ordered — together, they form the history of the universe.

Rule — a validation condition and/or effect on the State, associated with one or more Action/Event types.

State — not a separately stored entity. It is the result of applying, in order, all accepted Events over the base Universe generated from the seed:

SEED + RULES + EVENTS = CURRENT STATE

Two Nodes with the same seed, the same set of rules, and the same sequence of Events must arrive at the same State. This is the central contract that makes shared reality possible without any central authority.

4. Determinism and On-Demand Generation

4.1 The Determinism Contract

Given:

seed = X
rule_set = Y

any correct implementation of the protocol must generate the same base Universe (same initial Entities, Properties, and Relations) before any Event is applied. This allows Nodes to reconstruct parts of reality without needing to transfer the entire history — just the seed, plus the Events relevant to that region.

4.2 Spatial Locality

Since the universe is effectively unlimited (anything can be generated on demand from seed + coordinates), a client should never ask a Node “everything that exists.” Instead, it asks:

“what exists within radius R of coordinates (x, y, z)?”

The Node responds only with Entities and Events relevant to that region — generating on demand any base Entities not yet touched, and including only the Events actually recorded in that area. This keeps both processing and data transfer proportional to where the player actually is, not to the size of the entire universe.

4.3 Cascading Generation

Content generation within a universe follows a tree of causality organized in levels: no level decides a result on its own — each level only shapes the probability distribution available to the level below it.

Level 0 – Foundations (dimensions, time, causality, matter, energy, information, consciousness)
Level 1 – Physics (gravity, electromagnetism, thermodynamics, quantum behavior, entropy)
Level 2 – Metaphysics (magic, souls, psychic interaction – only possible if Level 0 allows it)
Level 3 – Cosmology (stars, galaxies, black holes)
Level 4 – Biology (probability of life, evolution, intelligence)
Level 5 – Civilization (technology, language, culture, economy)
Level 6 – History (the Event Record itself)

Each coordinate uses a deterministic resolution function:

`resolve(level, seed, coordinate, inherited_context) → Properties`

This function samples from a distribution shaped by parameters inherited from the level above — never a fixed “if X, then Y” table. Final Properties are the product of accumulated modifiers ($\text{value} = \text{base} \times \text{mod}(\text{axis}_1) \times \text{mod}(\text{axis}_2) \times \dots$), which allows every result to be explainable back to the observer.

4.4 Preview Without Divergence

An auxiliary function, `probability_of()`, executes exactly the same formula chain as `resolve()`, but stops before the final random sampling — returning the aggregated distribution instead of a rolled value. This guarantees that any probability shown to an observer never diverges from the actual result, because it is literally the same function, running in two modes.

5. Identity and History

5.1 Identity as a Fact of the Universe

The protocol does not define accounts, passwords, or logins. Identity is treated exactly like anything else: as an Event. The first time a given actor submits any Action to a Node, the Node produces an Event marking that actor’s entry into the universe. There is no server-side account table — an actor simply becomes real the moment it is first used, and that moment is permanently recorded like any other fact of the universe.

5.2 Conflicting Actions

Since a Node processes Actions one at a time, there is never ambiguity about which Action was accepted first when two actors target the same thing — whoever arrived first in the processing queue wins, and that becomes the recorded version. What matters is that the loser is informed why, with enough context to make sense of it — not just a generic rejection.

5.3 Narrative Artifacts in the World

Not every visit to a place needs to become a permanent Event — merely passing through a location without doing anything need not leave a trace. But an actor may choose to perform a deliberate action that does leave one: for example, recording a message at a coordinate. Once validated, that message becomes a normal Event, tied to that Entity/coordinate, and any client that visits that place afterward can retrieve and display it.

6. Multiverse and Portability

The initial version of the protocol assumed a single universe with a single authority. The protocol's extension generalizes this: there is no central authority Node. Each installation is a complete, self-sufficient Node. A participant generates their own universe locally from a seed (256 bits) and participates in it fully independently by default.

6.1 No Scarcity Model — Only Provenance

Universes and generated content are not scarce. What is unique is not “how many copies exist” — it is the Event history associated with a specific seed. Two instances generating the same seed obtain the same base universe, but potentially completely different Event histories.

6.2 Asynchronous, Optional Synchronization

Synchronization between participants is asynchronous, optional, and coordinated by the participants themselves — for example, two people agree, outside the protocol, to use the same seed, and later merge their Event Records. There is no server authority.

Rule: `conflict_resolution`

For any two Events claiming the same coordinate/Entity: the Event with the earlier universe-time wins. The losing side receives a compensating Event (e.g., "your base was affected by the merging of universes"), so that the conflict itself becomes a recorded, explainable fact — the same principle already used for conflicting actions within a single Node.

6.3 Capsules — Generic Portability Mechanism

Any Entity can be exported as a capsule: a self-contained payload, copyable and pasteable (compatible with pastebin, forums, text images), requiring no server at all.

Entity: `capsule<T>`

properties:

`payload_type, origin_seed, snapshot,`

```
provenance_events, signature (optional),  
redemption: { max_uses, redeemed_by }
```

A capsule shared publicly by default has `max_uses = 1` — once redeemed by one actor, subsequent redemption attempts are rejected. This prevents indefinite duplication of content from a single shared capsule, without requiring any central registry — verification is done against the local or merged Event Record itself.

7. Relative Power — A Use Case of the Protocol

An interesting example of how the protocol treats everything as relative to local context, rather than as an absolute value, is the treatment of an actor’s “power” within the universe.

Instead of an absolute value, an actor’s power is always calculated relative to the universe they are in:

```
relative_power = actor_attribute_value ÷  
                universe_reference_level
```

The `universe_reference_level` is derived from the same Level 0–1 axes that already generate the entire universe (energy, entropy, gravity) — it is not a new field, it is a reading of what already exists. Since the space of possible seeds is, in practice, unlimited, there always exists a rarer, more extreme universe whose reference level keeps pace with any actor’s growth — no actor is absolutely powerful relative to every universe that could ever be generated, only relative to the ones they have actually visited.

This is an example of how a simple design principle — “nothing is absolute, everything is relative to the universe’s local context” — propagates consistently through different parts of the protocol: from environmental compatibility, to economy, to an actor’s power.

8. What the Protocol Deliberately Does Not Define

To avoid overloading the specification’s scope, some points are left as future direction, not normative content:

- Live/synchronous synchronization mode between Nodes (direct P2P)
- Cryptographic identity stronger than “first actor observed”
- Rate limiting / abuse prevention (relevant only if a live network mode is added)
- Detailed physics beyond what is necessary to prove the concept — the protocol provides the Rule mechanism, not an extensive set of predefined physical laws

These omissions are deliberate: the protocol’s goal is not to specify every possible piece of content, but to provide a minimal, sufficiently generic foundation so that any content can be built on top of it by future implementations.

9. Why This Matters

Most “shared world” formats solve the consistency problem through a central authority: a server that decides what is true, and all clients trust it. Universe Node proposes an alternative:

consistency through mathematical determinism — if two parties agree on the seed, the rules, and the sequence of events, they agree on reality, without needing any intermediary authority.

This shifts the problem from “who is in charge of the world” to “what rules does the world follow” — a shift that makes possible an unlimited number of independent implementations, in any language, coexisting over the same shared reality, without any of them needing to trust the others beyond correctly following the same protocol.

Appendix — Quick Glossary

Actor — any entity capable of submitting an Action (player, system, another Node).

Causal cascade — the leveled generation mechanism, where each level only shapes the probability of the next level.

Determinism — the guarantee that the same seed + rules + events always produces the same State, in any correct implementation.

Node — a complete, self-sufficient instance that maintains and participates in a reality according to the protocol.

Seed — the 256-bit number that, together with the rule set, determines the base universe before any Event.

Physical Substrate — the mandatory, generic layer of Rules that every Event must respect, regardless of genre or fiction.

This document is released under CC BY-NC-ND 4.0 (Attribution-NonCommercial-NoDerivatives). It may be shared in unmodified form, with credit to the author, for non-commercial purposes. Full terms: <https://creativecommons.org/licenses/by-nc-nd/4.0/>

© 2026 Ozalune.